

DESARROLLO WEB EN ENTORNO SERVIDOR

Simulacro de Examen

NORMAS DE REALIZACIÓN

⚠ **El trabajo es estrictamente individual.** Prohibido el uso de herramientas de comunicación (Discord, WhatsApp, Telegram, correo, etc.). Si el profesor observa cualquiera de estas aplicaciones abiertas → calificación 0.

⚠ **Se permite internet, IA y apuntes personales.** NO se permite subir fotos del enunciado ni compartir el código.

⚠ **NO SE CALIFICARÁ:**

- Código generado por IA copiado y pegado sin adaptación a lo aprendido en clase: no se calificará el apartado entero, aunque funcione.
- Código incoherente, sobran te, no funcional o traído de proyectos ajenos (“código basura”): no se calificará el apartado entero, aunque funcione.

El objetivo de esta prueba evaluable es obtener evidencias de que el alumno ha aprendido a usar Spring Boot con los módulos, componentes y patrones de diseño trabajados en clase. No se calificará soluciones funcionales generadas por la IA, incluso mejores, pero ajenas a los contenidos impartidos.

⚠ **Los fuentes proporcionados deben modificarse o ampliarse exclusivamente en los puntos indicados por el enunciado.** No se permite reescribir, sustituir o reestructurar código existente que no haya sido solicitado. El proyecto entregado debe seguir siendo reconociblemente el proyecto original. Se anulará la calificación del apartado afectado.

⚠ **El proyecto debe compilar y ejecutarse para poder ser calificado.** Si hay código que no funciona, coméntalo antes de entregar.

⚠ **El profesor podrá solicitar defensa oral.** Sin comprensión demostrada la calificación será 0.

PROYECTO API REST DE PELÍCULAS [5 puntos]

Recibes el proyecto **Spring Boot del API REST de películas** trabajado en clase. En este caso la BD **H2 está en memoria**. El proyecto arranca correctamente pero presenta errores de comportamiento y funcionalidades incompletas que debes corregir y ampliar según las especificaciones de este enunciado.

La **arquitectura del proyecto y patrones** son los trabajados en clase a lo largo del curso.

1. AÑADIR UN ACTOR A UNA PELÍCULA [1 punto]

El endpoint que permite añadir un actor al reparto de una película presenta errores de comportamiento. El endpoint devuelve 201 pero la operación no llega a persistirse correctamente en base de datos. **Resuelve los siguientes bugs descritos en dos casos de prueba.**

CASO DE PRUEBA 1 [0,5 puntos]

El objeto `MovieCast` creado debe persistirse correctamente en base de datos y quedar en un estado consistente en memoria, con todas sus relaciones correctamente establecidas.

```
POST http://localhost:8081/api/movies/5/cast
Content-Type: application/json
```

```
{
  "actorId": 1,
  "characterName": "melola",
  "screenMinutes": 100,
  "salaryOverride": 1000
}
```

```
POST http://localhost:8081/api/movies/5/cast
Show Request
```

```
HTTP/1.1 201
Content-Length: 0
Date: Sun, 03 May 2026 18:04:19 GMT

<Response body is empty>
```

Actualmente no obtienes el nuevo personaje en las trazas por consola:

```
*****
** LISTADO DE PERSONAJES:
[MovieCast{movie=Dunkirk, actor=Cillian
Murphy, personaje=Tommy}]
```

Deberías obtener todos los personajes:

```
*****
** LISTADO DE PERSONAJES:

[MovieCast{movie=Dunkirk, actor=Leonardo
DiCaprio, personaje=melola},

MovieCast{movie=Dunkirk, actor=Cillian
Murphy, personaje=Tommy}]
*****
```

CASO DE PRUEBA 2 [0,5 puntos]

La detección de un actor duplicado en la misma película debe funcionar de forma fiable y lanzar una `ConflictException` con un mensaje descriptivo.

La detección de duplicados debe realizarse sobre la colección en memoria, no mediante consulta al repositorio.

```
POST http://localhost:8081/api/movies/1/cast
Content-Type: application/json
```

```
{
  "actorId": 1,
  "characterName": "melola",
  "screenMinutes": 100,
  "salaryOverride": 1000
}
```

Actualmente pueden suceder dos cosas:

- O no se detecta el actor duplicado. No da error, devuelve el código 201.
- O devuelve error 500.

Esta es la gestión correcta del duplicado que debes implementar (código 409):

```
HTTP/1.1 409
> (Headers) ...Content-Type: application/json...

{
  "timestamp": "2026-05-03T19:36:52.5896594",
  "status": 409,
  "error": "Conflict",
  "message": "El actor ya tiene un rol en esta película",
  "path": "/api/movies/1/cast"
}
```

2. RESEÑAS DE PELÍCULAS [3 puntos]

Se requiere implementar la funcionalidad de reseñas de películas. Una película puede tener múltiples reseñas; cada reseña pertenece a una única película.

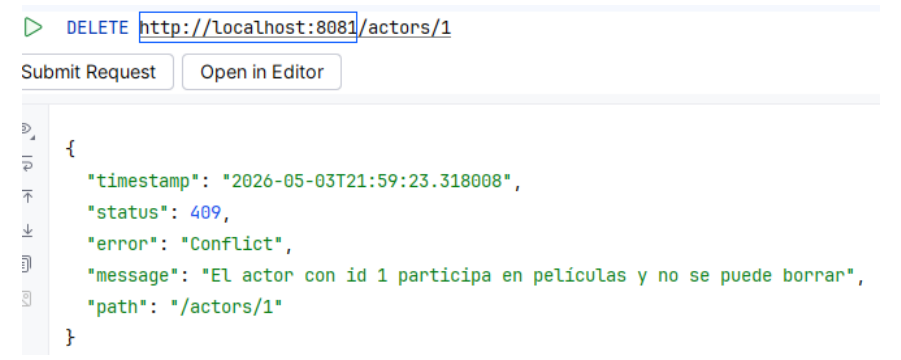
[0,5 puntos] Diseña la nueva entidad **Review** y establece la relación JPA bidireccional con **Movie** teniendo en cuenta las estrategias más apropiadas. **Script DDL de referencia:**

```
CREATE TABLE reviews (  
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,  
  author      VARCHAR(80)   NOT NULL,  
  body        VARCHAR(500)  NOT NULL,  
  rating      INT           NOT NULL,  
  created_at  DATE          NOT NULL,  
  active      BOOLEAN       NOT NULL DEFAULT TRUE,  
  movie_id    BIGINT        NOT NULL,  
  CONSTRAINT fk_review_movie FOREIGN KEY (movie_id) REFERENCES movies(id)  
);
```

Siguiendo estrictamente la arquitectura del proyecto implementa el siguiente endpoint:

POST <code>http://localhost:8081/movies/1/reviews</code> <i>Content-Type: application/json</i> <pre>{ "author": "John Doe", "body": "Una peli muy chula", "rating": 9 }</pre> • author: obligatorio, no vacío. • body: obligatorio, no vacío, máximo 500 caracteres. • rating: obligatorio, valor entre 1 y 10 inclusive.	[1,5 puntos] Si la operación es correcta → 201 Created. El cuerpo de la respuesta debe contener los datos de la reseña creada. <pre>HTTP/1.1 201 > (Headers) ...Content-Type: application/json... { "id": 2, "author": "John Doe", "body": "Una peli muy chula", "rating": 9, "createdAt": "2026-05-03", "movieId": 1, "movieTitle": "Inception" }</pre>
[0,5 puntos] Si la película no existe → 404 Not Found <pre>HTTP/1.1 404 (Headers) ...Content-Type: application/json... { "timestamp": "2026-05-03T21:10:11.8662508", "status": 404, "error": "Not Found", "message": "Película no encontrada con id: 10", "path": "/movies/10/reviews" }</pre>	[0,5 puntos] No cumplir las validaciones → 400 Bad Request <pre>HTTP/1.1 400 (Headers) ...Content-Type: application/json... { "timestamp": "2026-05-03T21:09:47.0782791", "status": 400, "error": "Bad Request", "message": "rating : La valoración máxima es 10", "path": "/movies/1/reviews" }</pre>

3. BORRADO DE UN ACTOR CON RESPUESTA ENRIQUECIDA [1 punto]

El endpoint que permite eliminar un actor del sistema devuelve actualmente un 409 Conflict con el mensaje de error estándar cuando el actor participa en alguna película.	 <pre> DELETE http://localhost:8081/actors/1 { "timestamp": "2026-05-03T21:59:23.318008", "status": 409, "error": "Conflict", "message": "El actor con id 1 participa en películas y no se puede borrar", "path": "/actors/1" } </pre>
Se requiere modificar el comportamiento para que la respuesta 409 incluya además la lista de películas en las que participa el actor.	 <pre> DELETE http://localhost:8081/actors/1 { "timestamp": "2026-05-03T22:01:09.7560759", "status": 409, "error": "Conflict", "message": "El actor participa en películas y no puede ser eliminado", "path": "/actors/1", "participaciones": [{ "movieId": 3, "movieTitle": "Interstellar", "characterName": "Cooper", "screenMinutes": 120 }, { "movieId": 1, "movieTitle": "Inception", "characterName": "Cobb", "screenMinutes": 110 }] } </pre>

Añadir actor al reparto → 1 punto

- Caso de prueba 1: estado consistente en memoria → 0,5 puntos
- Caso de prueba 2: detección de duplicados en memoria → 0,5 puntos

Reseñas de películas → 3 puntos

- Entidad Review + relación JPA bidireccional → 0,5 puntos
- Endpoint 201 Created con respuesta correcta → 1,5 puntos
- 404 Not Found si la película no existe → 0,5 puntos
- 400 Bad Request si no cumple validaciones → 0,5 puntos

Borrado de actor con respuesta enriquecida → 1 punto

PROYECTO MVC DE PELÍCULAS [5 puntos]

Se requiere implementar la **funcionalidad de favoritos de películas**. Cada usuario con rol USER puede guardar sus películas favoritas. **Si el usuario tiene rol ADMIN no tendrá esta funcionalidad**. Configura la **seguridad y visibilidad** de las funcionalidades en base al rol [0,5 puntos].

Los favoritos se almacenan en la **BD local** relacionando el usuario autenticado con el identificador de la película en la API remota.

Se te proporciona el esqueleto de la entidad y el script DDL de referencia [0,5 puntos] :

```
CREATE TABLE favorites (
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  movie_id    BIGINT NOT NULL,
  user_id     BIGINT NOT NULL,
  created_at  DATE    NOT NULL,
  CONSTRAINT fk_favorite_user FOREIGN KEY (user_id) REFERENCES app_users(id),
  CONSTRAINT uk_favorite UNIQUE (user_id, movie_id)
);
```

Se te proporciona también el esqueleto del controlador.

Implementa la lógica de negocio para las tres operaciones:

- [1 punto] **Añadir una película a favoritos desde el catálogo de películas.**
 - Si la película ya está en favoritos del usuario, la operación debe propagar una excepción y aparecerá un mensaje de aviso en la vista. [0,5 puntos]

The screenshot shows the 'Catálogo de Películas' view. At the top right, it says '6 películas'. A yellow box highlights a message: 'Esta película ya está en tus favoritos.' Below this is a table with columns: ID, TÍTULO, AÑO, GÉNERO, ESTADO, and a 'Ver' button with a star icon. The table lists two movies: 'Inception' (2010, SCI_FI, Activa) and 'The Dark Knight' (2010, ACTION, Activa). A yellow arrow points to the star icon in the 'Ver' button for 'Inception'.

ID	TÍTULO	AÑO	GÉNERO	ESTADO	Ver
1	Inception	2010	SCI_FI	Activa	Ver
2	The Dark Knight	2010	ACTION	Activa	Ver

- [1 punto] **Obtener los favoritos del usuario autenticado desde el panel de navegación.**

The screenshot shows the 'Cine App' navigation bar at the top. It has a 'Películas' tab and a 'Favoritos' tab, with 'Favoritos' being the active tab. Below the navigation bar is the 'Catálogo de Películas' view, which shows a table of movies. A yellow arrow points to the 'Favoritos' tab in the navigation bar. The table lists three movies: 'Inception' (2010, SCI_FI, Activa), 'The Dark Knight' (2010, ACTION, Activa), and 'Interstellar' (2010, SCI_FI, Activa). A yellow arrow points to the star icon in the 'Ver' button for 'Inception'.

ID	TÍTULO	AÑO	GÉNERO	ESTADO	Ver
1	Inception	2010	SCI_FI	Activa	Ver
2	The Dark Knight	2010	ACTION	Activa	Ver
3	Interstellar	2010	SCI_FI	Activa	Ver

Mis Favoritos

TÍTULO	AÑO	GÉNERO		
Interstellar	2014	SCI_FI		x
Dunkirk	2017	WAR		x
Inception	2010	SCI_FI		x

- **[1 punto] Eliminar un favorito.** Solo se puede eliminar un favorito que pertenezca al usuario autenticado.
 - **[0,5 puntos]** Tras el borrado exitoso de un favorito, debe mostrarse un mensaje informativo en la vista.

Mis Favoritos

TÍTULO	AÑO	GÉNERO		
Inception	2010	SCI_FI		x
The Dark Knight	2008	ACTION		x
The Prestige	2006	DRAMA		x

Mis Favoritos

Película eliminada de favoritos.

TÍTULO	AÑO	GÉNERO		
The Dark Knight	2008	ACTION		x
The Prestige	2006	DRAMA		x

- **Entidad + DDL** → 0,5 puntos
- **Seguridad y visibilidad** → 0,5 puntos
- **Añadir favorito** → 1 punto
- **Mensaje si ya existe** → 0,5 puntos
- **Obtener favoritos** → 1 punto
- **Eliminar favorito** → 1 punto
- **Mensaje flash tras borrado** → 0,5 puntos